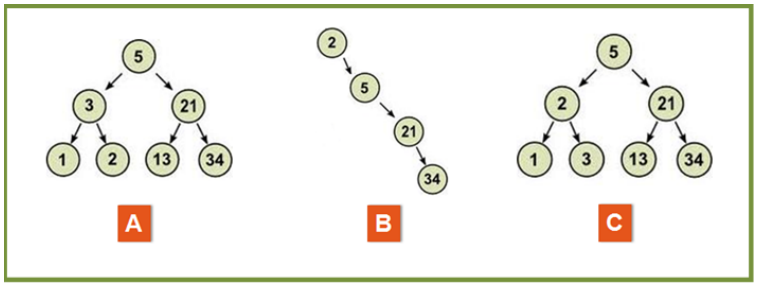
1. Árboles binarios de búsqueda. Implementación

Allan Weiss describe que un árbol es binario cuando cada nodo puede tener un máximo de dos hijos. Además, se dice que un árbol es binario de búsqueda si, para cada nodo X del árbol, los valores de todas las claves de su subárbol izquierdo son menores que la clave de X y los valores de las claves de su subárbol derecho son mayores.

Por ejemplo, en la siguiente figura los árboles B y C son árboles binarios de búsqueda, mientras que el árbol A no cumple con la definición, ya que el nodo con valor 2 está a la derecha del nodo con valor 3.

Figura 1:



2. Representación en Java

Para representar un árbol en Java, primero veamos cómo quedaría el diagrama de clases en donde identificamos 3 clases fundamentales: ArbolBinario, NodoBinario y Dato. La clase ArbolBinario contiene internamente al nodo raíz y expone públicamente operaciones asociadas al árbol, como Imprimir, Insertar, Buscar, etcétera. Contiene un nodo raíz del tipo NodoBinario.

La clase NodoBinario contiene el dato que se quiere almacenar en el árbol. La clase Dato implementa la interface Comparable, que permite comparar los datos almacenados entre sí para poder realizar las distintas operaciones. Además del dato, la clase NodoBinario contiene un nodo izquierdo y un nodo derecho del mismo tipo (NodoBinario). Esto permite definir la estructura de árbol como una estructura de datos recursiva.

A continuación, se muestra un ejemplo de cómo podrían representarse en Java las clases que describe el diagrama de clases mencionado anteriormente:

Figura 2:

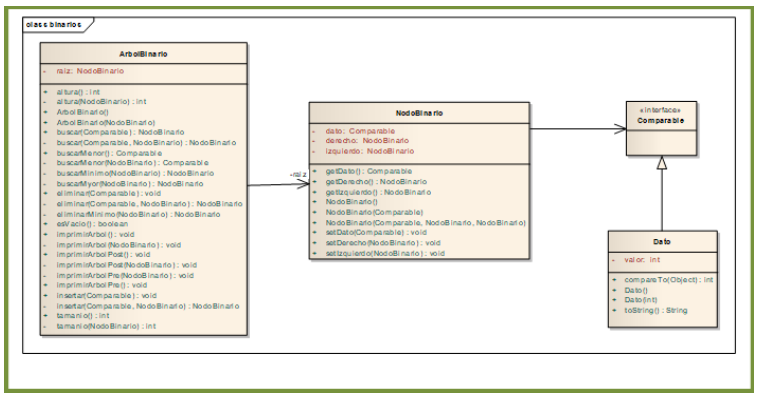


Figura 3:

```java
public class ArbolBinario {

    private NodoBinario raiz;

    public ArbolBinario() {
        this(null);
    }

    public ArbolBinario(NodoBinario raiz) {
        this.raiz = raiz;
    }

    public boolean esVacio() {
        return this.raiz == null;
    }

    public void imprimirArbol() {
        imprimirArbol(this.raiz);
    }
```

Figura 4:

```java
public class Dato implements Comparable {

    private int valor;

    public Dato() {
        this(0);
    }

    public Dato(int valor) {
        this.valor = valor;
    }

    @Override
    public int compareTo(Object o) {
        Dato nuevoDato = (Dato) o;
        if (this.valor > nuevoDato.valor) {
            return 1;
        } else if (this.valor < nuevoDato.valor) {
            return -1;
        } else {
            return 0;
        }
    }
```

Figura 5:

```java
public class NodoBinario {

    private Comparable dato;
    private NodoBinario izquierdo;
    private NodoBinario derecho;

    public NodoBinario(Comparable dato) {
        this.dato = dato;
    }

    public NodoBinario getDerecho() {
        return derecho;
    }

    public void setDerecho(NodoBinario derecho) {
        this.derecho = derecho;
    }

    public NodoBinario getIzquierdo() {
        return izquierdo;
    }

    public void setIzquierdo(NodoBinario izquierdo) {
        this.izquierdo = izquierdo;
    }

    public Comparable getDato() {
        return dato;
    }

    public void setDato(Comparable dato) {
        this.dato = dato;
    }
}
```

3. Recorridos

Se puede hacer un recorrido de un árbol en profundidad o en anchura. Los recorridos en anchura son por niveles; horizontalmente, se realizan desde la raíz a todos los hijos antes de pasar a la descendencia de alguno de los hijos.

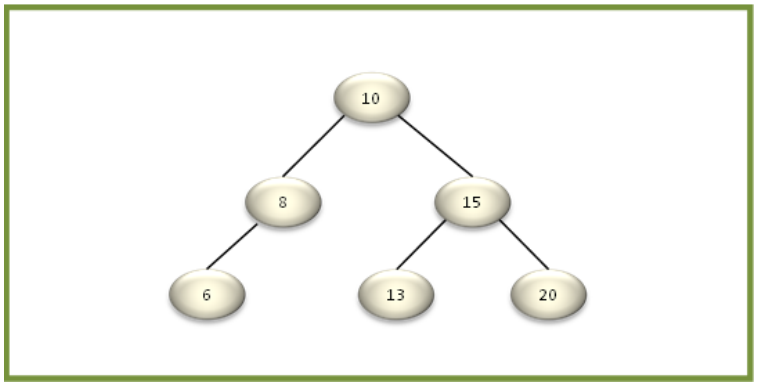
El costo de recorrer el árbol binario de búsqueda es O(*n*), ya que se necesita visitar todos los vértices.

El recorrido en profundidad lleva al camino desde la raíz hacia el descendiente más lejano del primer hijo y luego continúa con el siguiente hijo. Como recorridos en profundidad, tenemos *in-order, pre-order* y post order. En la siguiente figura, se puede ver un ejemplo de cómo quedarían los distintos recorridos para el siguiente árbol:

Tabla 1:

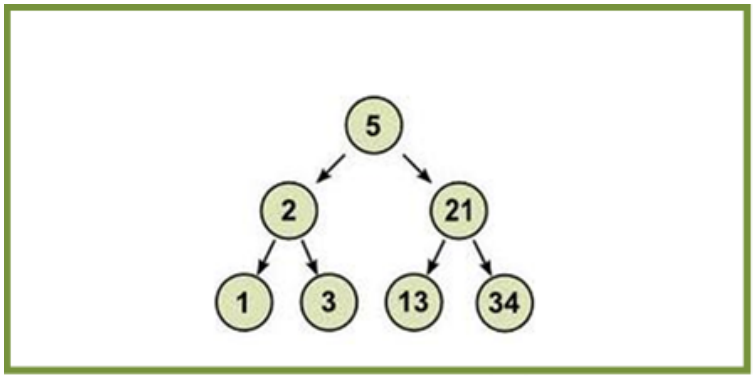
Recorrido Pre-orden (raíz, subárbol izq, subárbol der)	void preorder() **Imprimir** valor; **Si** hijo izquierdo != null preorder → izq; **Si** hijo derecho != null preorder → der;	**Salida:** 10, 8, 6, 15, 13, 20 **Costo**: O(N)
Recorrido Post-orden (subárbol izq, subárbol der, raíz)	void post-order() **Si** hijo izquierdo != null post-order → izq; **Si** hijo derecho != null post-order → der; **Imprimir** valor;	**Salida:** 6, 8, 13, 20, 15, 10 **Costo**: O(N)
Recorrido In-orden (subárbol izq, raíz subárbol der)	void inorder() **Si** hijo izquierdo != null inorder → izq; **Imprimir** valor; **Si** hijo derecho != null inorder → der;	**Salida:** 6, 8, 10, 13, 15, 20 **Costo**: O(N)

Figura 6:



Una propiedad de los árboles binarios de búsqueda es que, al hacer un recorrido en profundidad in-order, obtenemos los elementos ordenados de forma ascendente. Por ejemplo, para este árbol, el resultado de hacer el recorrido en in-order es: 1, 2, 3, 5, 13, 21, 34.

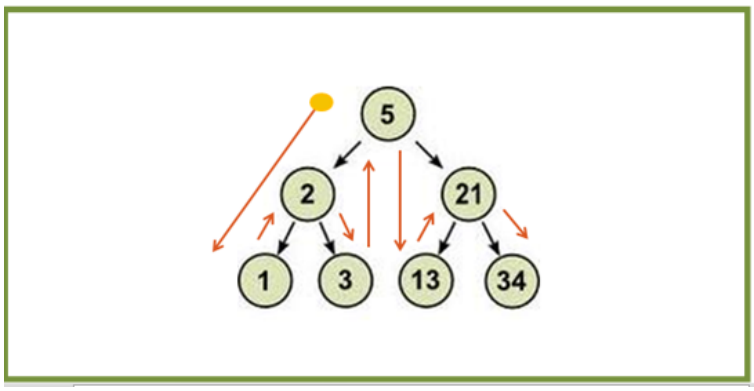
Figura 7:



El siguiente ejemplo de código muestra el algoritmo en Java para el recorrido in-order y la figura muestra cómo sería el recorrido del algoritmo paso por paso, es decir, cuándo se ejecuta el código:

Figura 8:

```java
public void imprimirArbol() {
    imprimirArbol(this.raiz);
}

private void imprimirArbol(NodoBinario nodo) {
    if (nodo != null) {
        imprimirArbol(nodo.izquierdo);
        System.out.print(nodo.dato.toString());
        imprimirArbol(nodo.derecho);
    }
}
```



Si bien el recorrido en in-order es el más usado, los recorridos pre-order y post-order son útiles en distintas situaciones. A continuación, se muestra la implementación del recorrido en pre-order y post-order respectivamente:

Figura 9:

```java
public void imprimirArbolPre() {
    this.imprimirArbolPre(this.raiz);
}

private void imprimirArbolPre(NodoBinario nodo) {
    if (nodo != null) {
        System.out.print(nodo.getDato().toString() + "  ");
        imprimirArbolPre(nodo.getIzquierdo());
        imprimirArbolPre(nodo.getDerecho());
    }
}
```

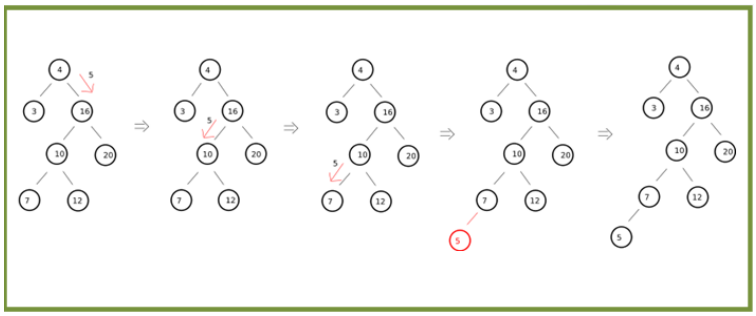
```java
public void imprimirArbolPost() {
    this.imprimirArbolPost(this.raiz);
}


private void imprimirArbolPost(NodoBinario nodo) {
    if (nodo != null) {
        imprimirArbolPost(nodo.getIzquierdo());
        imprimirArbolPost(nodo.getDerecho());
        System.out.print(nodo.getDato().toString() + "  ");
    }
}
```

4. Insertar

La inserción en un árbol binario se realiza siguiendo el criterio de búsqueda de este, es decir, si el valor que se quiere insertar es menor que el valor de la raíz, insertaremos el nodo en su subárbol izquierdo; de lo contrario, lo insertaremos en su subárbol derecho. Este algoritmo está representado a continuación.

La implementación en Java sería de la siguiente forma:

Figura 10:



5. Altura

La altura de un árbol corresponde a la cantidad de niveles que tiene y se define como el nivel del nodo de mayor nivel. Como cada nodo de un árbol puede considerarse a su vez como la raíz de un árbol, también podemos hablar de altura de ramas.

A continuación, se muestra la implementación en Java para calcular la altura:

Figura 11:

```java
public void insertar(Comparable x) throws Exception {
    this.raiz = insertar(x, raiz);
}


private NodoBinario insertar(Comparable x, NodoBinario nodo) throws Exception {
    if (nodo == null) {
        nodo = new NodoBinario(x, null, null);
    } else if (x.compareTo(nodo.getDato()) < 0) {
        nodo.setIzquierdo(insertar(x, nodo.getIzquierdo()));
    } else if (x.compareTo(nodo.getDato()) > 0) {
        nodo.setDerecho(insertar(x, nodo.getDerecho()));
    } else {
        throw new Exception("Elemento Duplicado");
    }
    return nodo;
}
```

```java
private int altura(NodoBinario nodo) {
    if (nodo == null) {
        return -1;
    }
    return Math.max(altura(
            nodo.getIzquierdo()), altura(nodo.getDerecho())) + 1;
}
```

Si el nodo es *null*, entonces la altura es -1. En cambio, debemos calcular la altura del subárbol izquierdo, la altura del subárbol derecho y obtener el máximo entre los dos subárboles. Cada vez que retornamos el máximo de cada subárbol, debemos incrementar 1, ya que estamos bajando un nivel en ambos lados.

6. Buscar

Una de las operaciones más importantes que tiene un árbol binario de búsqueda es el método Buscar, debido a que es la operación que permite acceder a un elemento de la estructura de datos en O(log *N*). A continuación, se muestra el método buscar implementado en Java:

Figura 12:

```java
public NodoBinario buscar(Comparable x) {
    return buscar(x, this.raiz);
}

private NodoBinario buscar(Comparable x, NodoBinario nodo) {
    if (nodo != null) {
        if (nodo.getDato().compareTo(x) > 0) {
            return buscar(x, nodo.getIzquierdo());
        } else if (nodo.getDato().compareTo(x) < 0) {
            return buscar(x, nodo.getDerecho());
        }
    }
    return nodo;
}
```

Lo primero que efectúa este método recursivo es verificar que el nodo pasado como parámetro sea distinto de *null*. Luego, compara el objeto del tipo Comparable *(x)* con el dato que está almacenado en el nodo y, si es menor, se llama al método buscar con el nodo izquierdo, ya que esto nos indica que el elemento que estamos buscando está en el subárbol izquierdo. En cambio, si el dato que está almacenado en el nodo es mayor a x, se llama al método buscar con el nodo derecho, ya que el elemento que estamos buscando se encuentra en el subárbol derecho.

7. Eliminar

Eliminar un nodo de un árbol es una de las operaciones más complejas y, tal como describe Allen Weiss (2000), podemos ver que hay 3 casos en los que para cada uno hay que aplicar una estrategia distinta:

Caso 1: en el caso en que el elemento que haya que eliminar sea una hoja, la eliminación es muy simple.

Caso 2: cuando el nodo que haya que eliminar tenga un solo hijo, hacemos que el padre del nodo que eliminar apunte a su único hijo.

Caso 3: cuando el nodo que haya que eliminar tenga 2 hijos, nos encontramos con el caso más complicado. En este caso, el nodo que eliminar debe ser reemplazado por el nodo más a la derecha en el subárbol izquierdo o por el nodo más a la izquierda en el subárbol derecho (es decir, por el máximo elemento de su subárbol izquierdo o por el mínimo elemento de su subárbol derecho).

Figura 13:

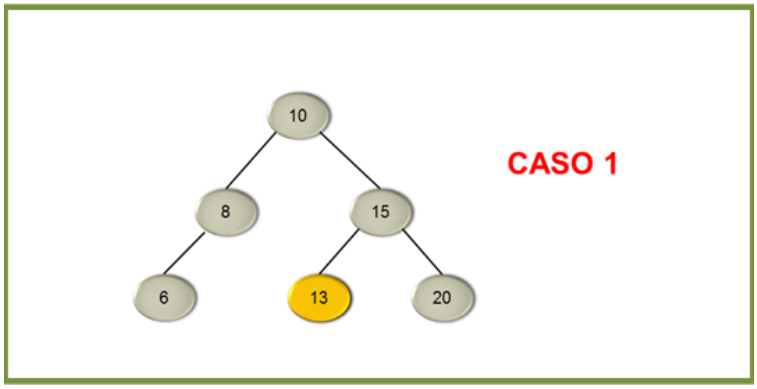


Figura 14:

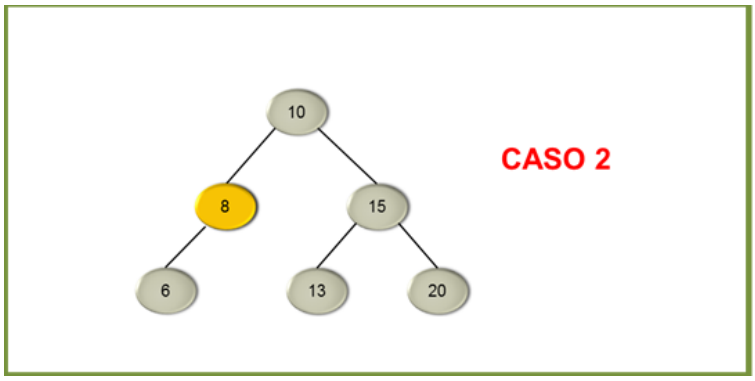
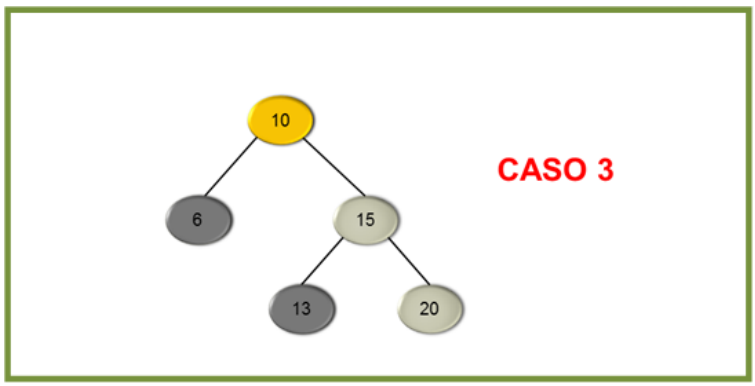


Figura 15:



Para la implementación en Java, es aconsejable tener implementados los métodos que busquen el mínimo y el máximo de un árbol, así como también métodos que eliminen el mínimo y el máximo de un árbol. Contando con estos métodos, es posible simplificar el algoritmo de eliminación para que se entienda más fácilmente.

A continuación, se muestran los métodos necesarios para realizar la eliminación de un nodo implementados en Java:

Figura 16:

```java
private NodoBinario buscarMinimo(NodoBinario nodo) {
    if (nodo != null) {
        while (nodo.getIzquierdo() != null) {
            nodo = nodo.getIzquierdo();
        }
    }
    return nodo;
}


private NodoBinario buscarMayor(NodoBinario nodo) {
    if (nodo != null) {
        while (nodo.getDerecho() != null) {
            nodo = nodo.getDerecho();
        }
    }
    return nodo;
}

private NodoBinario eliminarMinimo(NodoBinario nodo) {
    if (nodo.getIzquierdo() != null ) {
        nodo.setIzquierdo(eliminarMinimo(nodo.getIzquierdo()));
        return nodo;
    } else {
        return nodo.getDerecho();
    }
}

private NodoBinario eliminarMayor(NodoBinario nodo) {
    if (nodo.getDerecho() != null ) {
        nodo.setDerecho(eliminarMayor(nodo.getDerecho()));
        return nodo;
    } else {
        return nodo.getIzquierdo();
    }
}
```

Figura 17:

```java
private NodoBinario eliminar(Comparable x, NodoBinario nodo) {

     if (nodo != null) {
        if (nodo.getDato().compareTo(x) > 0 ) {
            nodo.setIzquierdo(eliminar(x, nodo.getIzquierdo()));
        } else {
            if (nodo.getDato().compareTo(x) < 0 ) {
                nodo.setDerecho(eliminar(x, nodo.getDerecho()));
            } else {
                if (nodo.getIzquierdo() != null && nodo.getDerecho() != null ) {
                    nodo.setDato(buscarMinimo(nodo.getDerecho()).getDato());
                    nodo.setDerecho(eliminarMinimo(nodo.getDerecho()));
                } else {
                    nodo = (nodo.getIzquierdo() != null) ? nodo.getIzquierdo() : nodo.getDerecho();
                }
            }
        }
     }
     return nodo;
}
```

Referencias

Cormen, T., Leiserson, C., Rivest, R. y Stein, C. (2002). *Introduction to Algorithms* (2nd ed.). McGraw Hill.

Sznajdleder, P. A. (2012). *Algoritmos a fondo con implementación en C y JAVA*. Buenos Aires: Alfaomega.

Weiss, M. A. (2000). *Estructuras de Datos en Java*. Buenos Aires, AR: Adisson Weisley.